Complete Configuration Guide: All Numbers Explained

Here's every configuration value in your project, why it's set that way, and the reasoning behind each number.

---

1. Kafka Consumer Configurations

From `application.yml` & `KafkaProperties.java`

```
ingester:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:10.10.61.51:9092,10.10.6
```

Config	Value	Why
bootstrap-servers	3 Kafka brokers	✅ **High availability** - If one broker dies, others keep working. 3 is the minimum for HA (majority quorum)

```
poll-timeout-ms: 1000
```

Config	Value	Why
poll-timeout-ms	1000ms (1 second)	✅ **Balanced responsiveness** - Short enough to react to shutdown quickly, long enough to not waste CPU. 1 second is the sweet spot.

```
max-poll-records: 5000
```

Config	Value	Why
max-poll-records	5000	✅ **Batch efficiency** - Kafka recommends 500-5000 for good throughput. 5000 = high throughput without overloading memory. Each record ~500 bytes → 2.5MB per poll.

```
session-timeout-ms: 30000
```

Config	Value	Why
session-timeout-ms	30000ms (30 seconds)	✅ **Crash detection** - If consumer stops sending heartbeats, Kafka assumes it's dead after 30s. Long enough to handle GC pauses, short enough to detect real failures.

```
heartbeat-interval-ms: 3000
```

Config	Value	Why
heartbeat-interval-ms	3000ms (3 seconds)	✅ **Keep alive** - Heartbeats every 3s keep consumer registered. 3s is 1/10 of session timeout (30s) - standard ratio.

```
max-poll-interval-ms: 300000
```

Config	Value	Why
max-poll-interval-ms	300000ms (5 minutes)	✅ **Processing window** - If consumer takes >5min to process records, Kafka assumes it's dead. 5 minutes gives time for slow HDFS writes.

```
enable-auto-commit: false
```

Config	Value	Why
enable-auto-commit	false	✅ **Zero data loss** - Manual commit ensures offsets are only committed AFTER HDFS write succeeds. Critical for production.

```
auto.offset.reset: earliest  (via additional-configs)
```

Config	Value	Why
auto.offset.reset	earliest	✅ **Start from beginning** - When a new consumer group starts, read all existing data. `earliest` = no data loss.

```
consumer-instances: ${CONSUMER_INSTANCES:1}
```

Config	Value	Why

Config	Value	Why
consumer-instances	1 (dev), 3 (prod)	✅ **Horizontal scaling** - Match number of partitions (3). More consumers = higher throughput, but max is number of partitions.

---

2. Rolling/Batch Configurations

```
rolling:
  max-record-count: ${ROLLING_MAX_RECORD_COUNT:8000000}
```

Config	Value	Why
max-record-count	8,000,000 records	⚠️ **High throughput batch** - This is VERY large. With 500 bytes/record = 4GB batch! ✅ **Reason:** Your 3 partitions × 100MB = 300MB total. 8M records handles this. ⚠️ **Risk:** Memory pressure. Consider lowering to 500,000.

```
max-size-bytes: ${ROLLING_MAX_SIZE_BYTES:805306368}
```

Config	Value	Why
max-size-bytes	805MB	✅ **Matches count** - 8M records × 500 bytes = 4GB, but size limit is 805MB. **Why 805MB?** HDFS block size default is 128MB. 805MB = ~6 blocks. Good for batch size.

```
max-age: ${ROLLING_MAX_AGE:2m}
```

Config	Value	Why
max-age	2 minutes	✅ **Time-based flush** - Even if count/size not reached, flush every 2 minutes. Prevents data sitting in memory too long.

---

3. HDFS Configurations

```
storage:
  parquet:
    compression-codec: ${PARQUET_COMPRESSION:SNAPPY}
```

Config	Value	Why

Config	Value	Why
compression-codec	SNAPPY	✅ **Best tradeoff** - Fast compression (CPU), good compression ratio (size). Better than GZIP (slow) or no compression (storage waste).

```
block-size-bytes: ${PARQUET_BLOCK_SIZE:134217728}
```

Config	Value	Why
block-size-bytes	128MB	✅ **HDFS optimal** - HDFS default block size is 128MB. Matches HDFS block size for optimal performance.

```
page-size-bytes: ${PARQUET_PAGE_SIZE:1048576}
```

Config	Value	Why
page-size-bytes	1MB	✅ **Parquet default** - Page size = unit of compression. 1MB is Parquet's default, good for most workloads.

```
enable-dictionary: ${PARQUET_ENABLE_DICTIONARY:true}
```

Config	Value	Why
enable-dictionary	true	✅ **Space savings** - Dictionary encoding compresses repeated values (like IP addresses). Massive space savings for Netflow data.

---

4. HDFS HA Configuration

```
storage:
  hdfs:
    nameservice: team1
    nodes:
      team1-node1: ${HDFS_NODE1:10.10.61.51:8020}
      team1-node2: ${HDFS_NODE2:10.10.61.52:8020}
      team1-node3: ${HDFS_NODE3:10.10.61.53:8020}
```

Config	Value	Why
nameservice	team1	✅ **HA NameNode** - Logical name for the HDFS cluster.

Config	Value	Why
3 NameNodes	3 nodes	✅ **High availability** - If one NameNode fails, others take over. 3 = majority quorum.
Port 8020	8020	✅ **HDFS RPC port** - Default HDFS NameNode RPC port.

---

5. JVM & Docker Configurations

```
ENV JAVA_OPTS="-Xmx512m -Xms256m"
```

Config	Value	Why
-Xmx512m	512MB max heap	✅ **Memory limit** - 8M records × 500 bytes = 4GB max. But with 805MB size limit, 512MB heap is safe. ⚠️ **Risk:** If `max-size-bytes` is 805MB and records are 500 bytes, memory could exceed 512MB. Consider lowering batch size.
-Xms256m	256MB initial heap	✅ **Performance** - Pre-allocates memory, avoids GC pauses during startup.

---

6. Docker Container Configurations

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3
```

Config	Value	Why
interval	30s	✅ **Liveness probe** - Check every 30 seconds. Frequent enough, not too frequent.
timeout	10s	✅ **Health check timeout** - Health check must complete in 10 seconds.
start-period	60s	✅ **Startup grace** - Give 60 seconds for app to start before health checks count as failures.
retries	3	✅ **Failure tolerance** - 3 failures before container is considered unhealthy.

---

7. Test Configurations

```
rolling:
  max-record-count: 5  # In StorageIT
```

Config	Value	Why
max-record-count	5 (test)	✅ **Quick test** - Small batch size makes tests fast.

---

8. Server Architecture

Server	Role	Port
`10.10.61.51`	Kafka Broker + HDFS NameNode	9092 (Kafka), 8020/9870 (HDFS)
`10.10.61.52`	HDFS NameNode	8020
`10.10.61.53`	Kafka Broker + HDFS NameNode	9092, 8020
`10.10.61.54`	Kafka Broker	9092

Summary Table: All Critical Values

Config	Value	Risk Level	Reason
`max-poll-records`	5000	Low	Standard Kafka practice
`session-timeout-ms`	30000	Low	Industry standard
`enable-auto-commit`	false	Low	Required for zero data loss
`max-record-count`	8,000,000	**HIGH**	Memory risk - consider 500,000
`max-size-bytes`	805MB	Medium	Memory risk with 8M count
`Xmx`	512MB	Medium	Low for 8M records
`consumer-instances`	3	Low	Matches partitions
`block-size-bytes`	128MB	Low	HDFS default

⚠️ Recommendation: Reduce Batch Size

```
rolling:
  # Current values (High Risk):
  max-record-count: 8000000   # 8M records = memory risk
  max-size-bytes: 805306368   # 805MB = memory risk

  # Recommended values (Safe):
  max-record-count: 500000    # 500K records = ~250MB
  max-size-bytes: 268435456   # 256MB = safe for 512MB heap
  max-age: 2m                 # Keep as-is
```

Why: With `-Xmx512m`, a batch of 8M records could exceed memory. 500K records is safe and still high throughput.